



Felvételi, 2023. szeptember
– Alapképzés, informatika vizsga –

Hivatalból 1 pont jár. Munkaidő 3 óra.

Megjegyzések:

A pszeudokód algoritmusban

- az „=” értékadást jelent
- az „==” egyenőségvizsgálatot jelent
- a „**minden** $i = 1, n, 1$ **végezd**” jelentése: minden $i = 1, 2, 3, \dots, n$ értékre végezd el...
- Minden egész érték tárolható integer/int típusú változóban.
- A bemeneti adatok helyesnek tekinthetők.
- A programokat megjegyzésekkel kell ellátni! Törekedjünk hatékony megoldásokra! (maximális pontszám feltétele)

1. (1 pont) Mi kerül kiírásra az alábbi pszeudokód algoritmusrészlet nyomán, amennyiben $n = 5$?

```
c = 1
┌ amíg n > 0 végezd
|   a = n + 1
|   c = c + a
|   n = n - 1
└─
kiír c
```

21

2. (1 pont) Mit ír ki az alábbi pszeudokód algoritmusrészlet, ha $n = 5$ és $a[1..n] = \{1, 6, 9, 2, 5\}$?

```
p1 = 1
p2 = 0
┌ minden i = 1, n, 1 végezd
|   ┌ ha a[i] % 3 == 0 akkor
|   |   p1 = p1 * (a[i] / 3)
|   |   különben
|   |       p2 = p2 + (a[i] % 3)
|   └─
└─
kiír p1, p2
```

6 5

3. (1 pont) Írj Pascal vagy C/C++ **programot**, amely beolvasson a billentyűzetről két különböző, pontosan kétszámjegyű természetes számot, majd kiírja a két szám közül a vezérszámot. Két szám közül az a vezérszám, amelynek az első számjegye szigorúan nagyobb, mint a másik szám első számjegye és a számjegyeinek az összege is szigorúan nagyobb, mint a másik szám számjegyeinek az összege. Ha mindkét feltétel egyik szám esetén sem teljesül, akkor írd ki a „NINCS VEZERSZÁM” üzenetet.

PÉLDÁK

INPUT OUTPUT

13 47 47

INPUT OUTPUT

56 91 NINCS VEZERSZAM

```
int main()
{
    int a, b;
    cin >> a >> b;
    if( a/10 > b/10 && (a/10 + a%10) > (b/10 + b%10) ) { cout << a; }
    else if ( b/10 > a/10 && (b/10 + b%10) > (a/10 + a%10) ) { cout << b; }
    else { cout << "NINCS VEZERSZAM"; }
    return 0;
}
```

4. (1 pont) Írj Pascal vagy C/C++ **programot**, amely beolvas a billentyűzetről egy n ($4 \leq n \leq 100$), természetes számot. A program írja ki külön sorokba az n -nél szigorúan kisebb négyzetszámokat.

PÉLDA

INPUT OUTPUT

16 1
 4
 9

```
int main()
{
    int n;
    cin >> n;
    int i = 1;
    do{
        cout << i*i << " ";
        ++i;
    }while( i*i < n );
    return 0;
}
```

5. (1 pont) Írj Pascal vagy C/C++ **programot**, amely beolvas a billentyűzetről egy n természetes számot, majd egy n elemű természetes számokat tartalmazó számsorozatot, és kiírja a képernyőre, hogy a számsorozat hány eleme esetében egyezik meg a sorszám paritása az érték paritásával (a sorszám is páros és az érték is páros, vagy a sorszám is páratlan és az érték is páratlan). (Megjegyzés: a sorszámok 0-tól indulnak).

PÉLDA

INPUT OUTPUT

5
122 23 33 44 334

3

Magyarázat: a kiemelt pozíciókon egyezik meg a sorszám és az érték paritása.

```
int main()
{
    int n, szam, k = 0;
    cin >> n;
    for( int i = 0 ; i < n ; ++i ){
        cin >> szam;
        if( i%2 == szam%2 ){ ++k; }
    }
    cout << k;
}
```

```
    return 0;
}
```

6. (1 pont) Írj Pascal vagy C/C++ **programot**, amely beolvas a billentyűzetről egy ötszámjegyű természetes számot, és kiírja külön-külön sorba a beolvasott szám számjegyeiből alkotott számokat lépésenként, először jobbról-balra, majd balról-jobbra haladva (lásd a példát).

PÉLDA

INPUT
45975

OUTPUT

5
57
579
5795
57954
4
45
459
4597
45975

```
int main()
{
    int a, b = 0;
    cin >> a;
    while( a ){
        b = b * 10 + a % 10;
        cout << b << endl;
        a /= 10;
    }
    while( b ){
        a = a * 10 + b % 10;
        cout << a << endl;
        b /= 10;
    }
    return 0;
}
```

7. (1 pont) Írj Pascal vagy C/C++ **függvényt**, amely paraméterként kap egy n természetes számot és egy egydimenziós tömbben tárolt n elemű számsorozatot. A függvény eldönti, hogy a számsorozat szimmetrikus-e abban az értelemben, hogy az i . elemének a számjegyszáma megegyezik az $(n-1-i)$. elemének a számjegyszámával (az elemeket 0-tól indexeltük). A függvény térítsen vissza **1**-et, amennyiben a válasz igen, és **0**-át, amennyiben a válasz nem.

PÉLDÁK

$n = 4$

számsorozat: 112 28 13 841

válasz: **igen**

$n = 5$

számsorozat: 1 12 1468 27 11

válasz: **nem**

```
int szamjegyszam( int szam ){
    int szjsz = 0;
    while( szam ){
        ++szjsz;
        szam /= 10;
    }
    return szjsz;
}

int szimmetrikus( int n, int *a ){
    for( int i = 0, j = n-1 ; i < j ; ++i, --j ){
```

```

        if( szamjegyszam(a[i]) != szamjegyszam(a[j]) ){
            return 0;
        }
    }
    return 1;
}

```

8. (1 pont) Írj Pascal vagy C/C++ **programot**, amely a *matrix.in* állományból beolvassa az n természetes számot ($4 \leq n \leq 100$), valamint egy $n \times n$ méretű mátrixot. Írassuk ki a mátrix elemeit az ábrán megadott sorrendben:

1	2	3	4	11
8	9	5	12	22
10	6	13	21	25
7	14	20	24	23
15	19	18	17	16

Az ábrán a számok azt mutatják ($n = 5$ esetre), hogy milyen sorrendben kell kiírni a mátrix elemeit:

- Mellék-átló felett: 1. sorban jobbra, majd a mellék-átló feletti átlón le; következő soron jobbra, majd a következő mellékátlóval párhuzamos átlón le; stb.
- Mellék-átlón: a mellékátlón le.
- Mellék-átló alatt: utolsó soron balra, majd a mellék-átló alatti átlón fel; utolsó előtti soron balra, majd a következő mellékátlóval párhuzamos átlón fel; stb.

PÉLDA

INPUT

```

4
1 2 4 5
8 9 0 5
2 8 6 6
7 3 5 9

```

OUTPUT

```

1 2 4 9 2 8 5 0 8 7 9 5 3 6 5 6

```

```

void mellekatlo_felett( int , int (*)[100] );
void mellekatlon_le( int , int (*)[100] );
void mellekatlo_alatt( int , int (*)[100] );
int main()
{
    ifstream fin("matrix.txt");
    int n;
    fin >> n;
    int a[100][100];
    for( int i = 0 ; i < n ; ++i ){
        for( int j = 0 ; j < n ; ++j ){
            fin >> a[i][j];
        }
    }
    mellekatlo_felett(n,a);
    mellekatlon_le(n,a);
    mellekatlo_alatt(n,a);
    return 0;
}

void mellekatlo_felett( int n , int (*)[100] ){
    int i,j,k=n-2; //k az atlo menti indexosszeg
    for( int m = 0 ; m < n/2 ; ++m ){ // m "menetunk" van
        i = m; j = 0;
        while( i+j <= k ){ cout << a[i][j] << " "; ++j; } //vizzszintes
    }
}

```

```

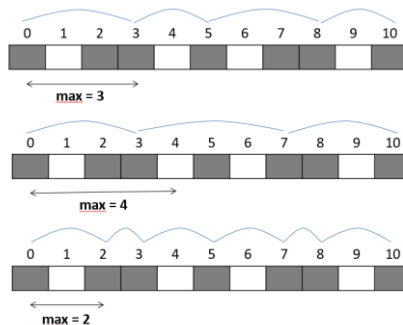
        ++i; j = k - i;
        while( j >= 0 ){ cout << a[i][j] << " "; ++i; --j; } //atlos
        --k;
    }
}

void mellekatlon_le( int n , int a[][100] ){
    for( int j = n-1 ; j >= 0 ; --j ){
        cout << a[n-1-j][j] << " ";
    }
}

void mellekatlo_alatt( int n , int a[][100] ){
    int i,j,k = n; //k az atlo menti indexosszeg
    for( int m = 0 ; m < n/2 ; ++m ){ // m "menetunk" van
        i = n-1-m; j = n-1;
        while( i+j >= k ){ cout << a[i][j] << " "; --j; } //vizszintes
        --i; j = k - i;
        while( j < n ){ cout << a[i][j] << " "; --i; ++j; } //atlos
        ++k;
    }
}

```

9. (1 pont) Írj Pascal vagy C/C++ **függvényt**, amely paraméterként megkapja az n természetes számot ($1 < n < 101$), egy egydimenziós tömbben tárolt n elemű bináris számsorozatot (az első és utolsó elem garantáltan 1-es), valamint egy további pozitív egész értéket (legyen x). A bináris számsorozat egy megsérült pallót kódol, amely esetében az 1-es értékek pozícióiban van deszka, a 0-ás értékek pozícióiban pedig nincs. A x érték jelenti, hogy maximum hány pozíciót tud ugrani egy kőszáli kecske. A függvény térítse vissza, hogy mennyi a minimális számú ugrás, amellyel a kecske át tud jutni a pallón. Ha nem tud átmenni, akkor a visszatérített érték legyen 0.



PÉLDA

INPUT

11

1 0 1 1 0 1 0 1 1 0 1

3

OUTPUT

4

Magyarázat: a fenti ábra első esetére vonatkozik a példa, amikor a palló hossza 11, és maximum 3 deszkát tud ugrani a kecske. Ez esetben minimum 4 ugrásból tud átmenni, például az ábra szerinti deszkákra ugorva (0., 3., 5., 8., 10.)

```

int ugrasszam( int n, int *a, int x ){
    int i = 0, j, k = 0;
    do{
        j = i + x;
        if( j >= n-1 ){ return k+1; }
        while( a[j] == 0 ){ --j; }
        if( j == i ){ return 0; }
        i = j; ++k;
    }while( 1 );
}

```

